

Struct como TAD

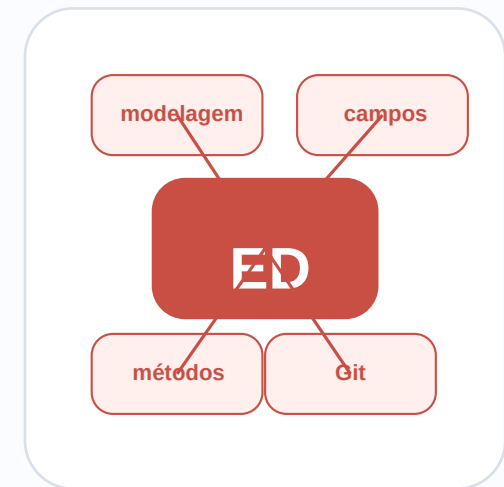
Roteiro organizado para modelar structs, escolher dados coerentes e implementar operações.

modelagem

campos

métodos

Git



Objetivo

Criar uma struct para cada contexto, escolhendo dados coerentes e operações adequadas.

Entrega

Criar repositório Git, fazer commits por exercício e enviar o link na tarefa.

Como estudar

Use cada contexto como um pequeno TAD: dados + operações + teste no Main.

JogadorFutebol

Modele um jogador com dados esportivos e operações de controle de cartões e vínculo.

Contexto

Futebol

Dados possíveis

- ☐ Nome
- ☐ Idade
- ☐ Posição
- ☐ Número da camisa
- ☐ Clube
- ☐ Cartões amarelos
- ☐ Cartões vermelhos
- ☐ Status de vínculo

Operações

- ☐ RegistrarCartaoAmarelo()
- ☐ RegistrarCartaoVermelho()
- ☐ VerificarVinculoClube()
- ☐ Imprimir()

EquipeEsports

Modele uma equipe de e-sports com histórico de campeonatos e premiações.

Contexto

E-sports

Dados possíveis

- ☐ Nome da equipe
- ☐ Jogo principal
- ☐ País
- ☐ Ano de estreia
- ☐ Campeonatos vencidos
- ☐ Valor total de premiações

Operações

- ☐ RegistrarCampeonatoVencido(valorPremio)
- ☐ AtualizarValorTotalPremiacoes(valor)
- ☐ VerificarAnoEstreia()
- ☐ Imprimir()

Produto

Modele um produto com preço, estoque e regras de desconto.

Contexto

Estoque e venda

Dados possíveis

- ☐ Nome
- ☐ Código
- ☐ Categoria
- ☐ Preço
- ☐ Quantidade em estoque
- ☐ Marca
- ☐ Disponibilidade

Operações

- ☐ AplicarCupomDescontoValor(valor)
- ☐ AplicarCupomDescontoPorcentagem(percentual)
- ☐ VerificarQuantidadeEmEstoque()
- ☐ Imprimir()

Professor

Modele um professor com salário, faltas e carga horária de trabalho.

Contexto

Professor

Dados possíveis

- ☐ Nome
- ☐ Disciplina
- ☐ Titulação
- ☐ Salário
- ☐ Carga horária
- ☐ Quantidade de faltas
- ☐ Departamento

Operações

- ☐ ReajustarSalarioEmValor(valor)
- ☐ ReajustarSalarioEmPorcentagem(percentual)
- ☐ DescontarFaltaEmValor(valor)
- ☐ DescontarFaltaEmPorcentagem(percentual)
- ☐ AumentarCargaHoraria(horas)
- ☐ Imprimir()

Como resolver cada exercício

Use o mesmo roteiro para todos os contextos e mantenha commits pequenos.

1. Campos

1

Liste todos os dados antes de programar a struct.

2. Métodos

2

Crie uma função para cada operação pedida.

3. Teste

3

No Main, preencha a struct e chame todos os métodos.

4. Commit

4

Faça commits separados, por exemplo: exercicio-jogador e exercicio-produto.